

Section 1: Edge Impulse Results

Edge Impulse Link: <https://studio.edgeimpulse.com/public/1074047/live>

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

Raw inputs are accelerometer values accX, accY and accZ.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

There are 33 input features. With respect to each accelerometer dimension (accX, accY, accZ), we have the following 11 features

- RMS (e.g. accX RMS)
- Peak 1 Height (e.g. accX Peak 1 Height)
- Peak 1 Freq (e.g. accX Peak 1 Freq)
- Peak 2 Height (e.g. accX Peak 2 Height)
- Peak 2 Freq (e.g. accX Peak 2 Freq)
- Peak 3 Height (e.g. accX Peak 3 Height)
- Peak 3 Freq (e.g. accX Peak 3 Freq)
- Spectral Power 0.1 - 0.5 Hz (e.g. accX Spectral Power 0.1 - 0.5 Hz)
- Spectral Power 0.5 - 1.0 Hz (e.g. accX Spectral Power 0.5 - 1.0 Hz)
- Spectral Power 1.0 - 2.0 Hz (e.g. accX Spectral Power 1.0 - 2.0 Hz)
- Spectral Power 2.0 - 5.0 Hz (e.g. accX Spectral Power 2.0 - 5.0 Hz)

3. NN classifier outputs: State the outputs of the NN classifier.

- Nominal
- Off

4. *Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.*

I used the following seven spectral features for the anomaly detector.

- accX RMS
- accX Peak 1 Height
- accY Peak 3 Height
- accX Peak 1 Freq
- accY RMS
- accY Peak 2 Freq
- accY Peak 1 Height

These were the most important features as shown below



Figure 1: Feature Importance

5. *Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.*

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.248545
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 0.272231
Starting inferencing in 2 seconds...
```

Figure 2: Inference Without Motion

```
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.007813
  off: 0.992187
Anomaly prediction: 61.308697
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.000000
  off: 0.996094
Anomaly prediction: 38.826378
Starting inferencing in 2 seconds...
[]
```

Figure 3: Inference With Motion

6. *Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.*

The Anomaly Prediction score goes up in value significantly when I move the Arduino board around. The classifier is good at detecting changes with respect to accX and accY values since these are the most important features. However, the classifier would not be good at detecting changes with respect to accZ values since none of the accZ attributes were deemed to be important enough features compared to accX and accY. When deploying this detector in the real world, one must take note of the direction the x, y and z axes are pointing. In addition, the microcontroller will still be subject to the laws of motion. False anomalies could be detected if the fan is running while being transported in a truck or in an elevator.

Section 2: Autoencoder Model Results

Visualizations of Training Data

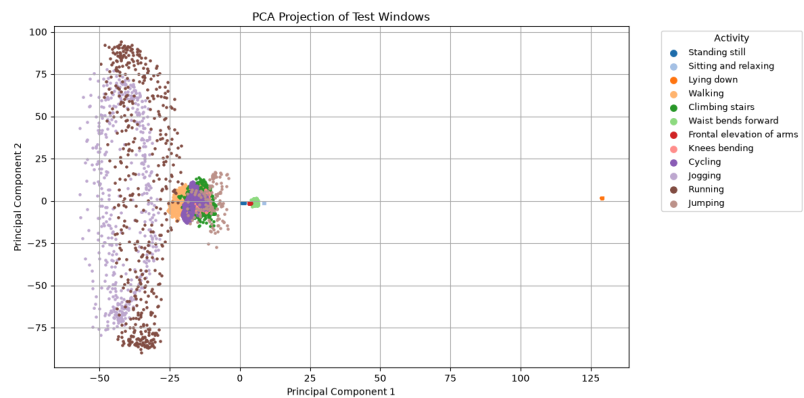


Figure 4: PCA

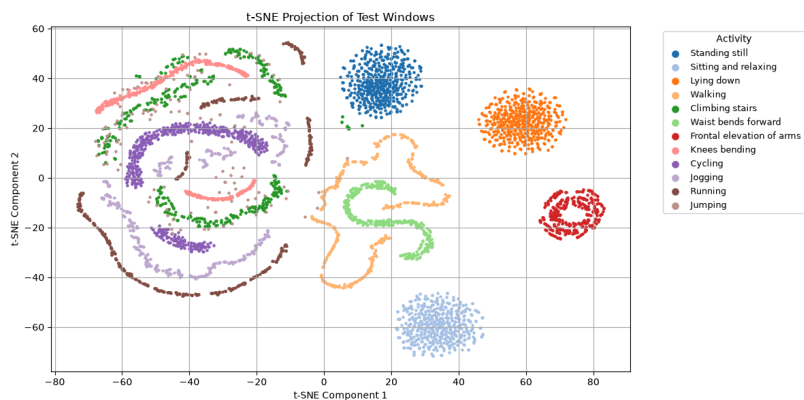


Figure 5: t-SNE

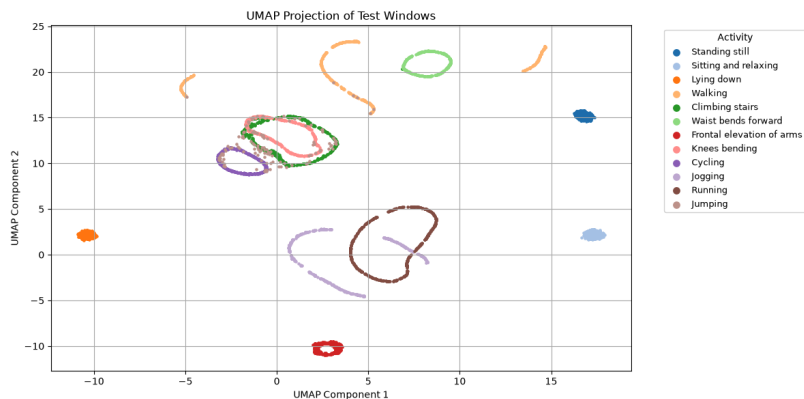


Figure 6: UMAP

Training and Validation Loss Curves

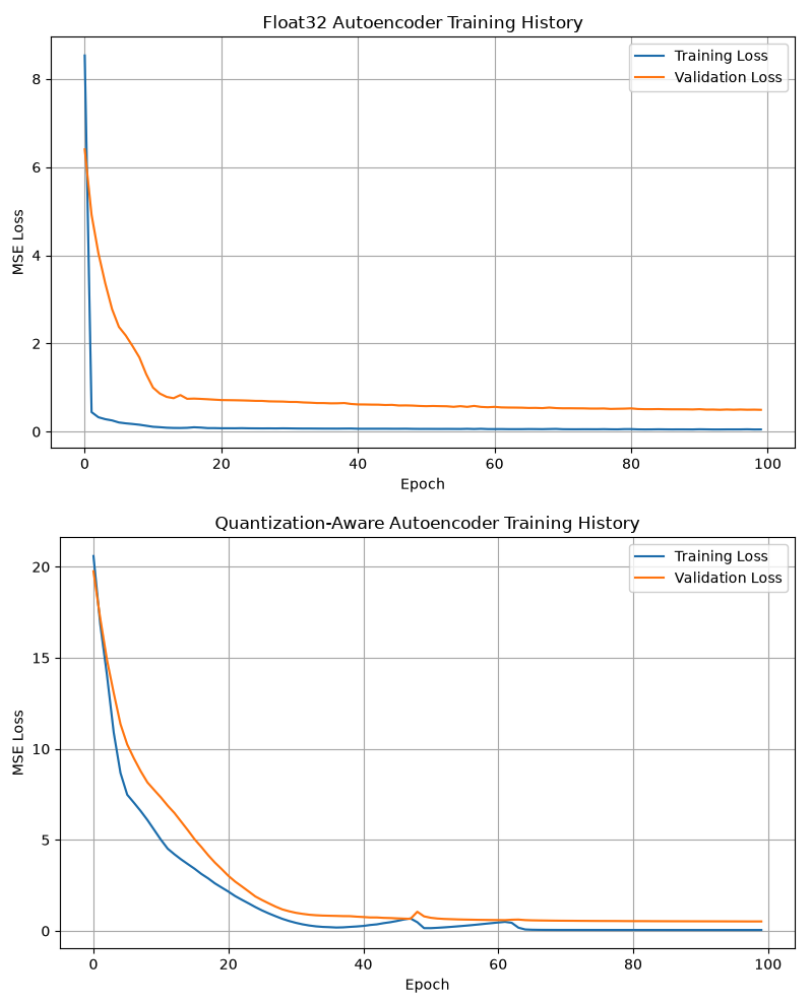


Figure 7: Float32 and Quantization-Aware Autoencoder Training and Validation Loss Curves

Reconstruction Error



Figure 8: Reconstruction Error Distribution

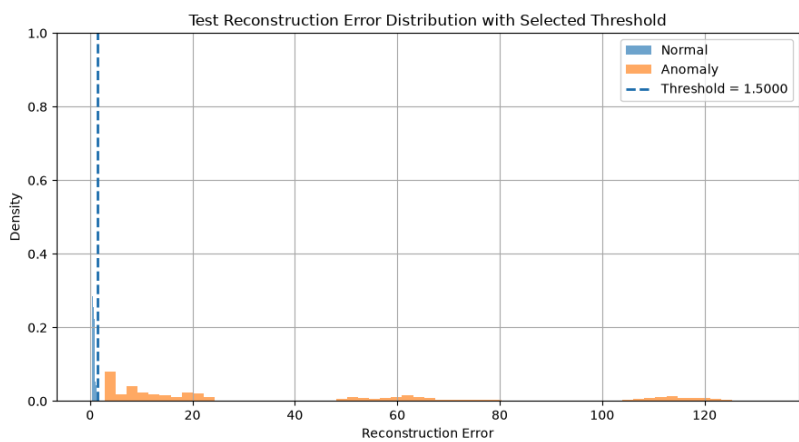


Figure 9: Reconstruction Error Distribution With Anomaly Threshold

Autoencoder Test Evaluation

Train set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350
Test set evaluation				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Figure 10: Classification Report

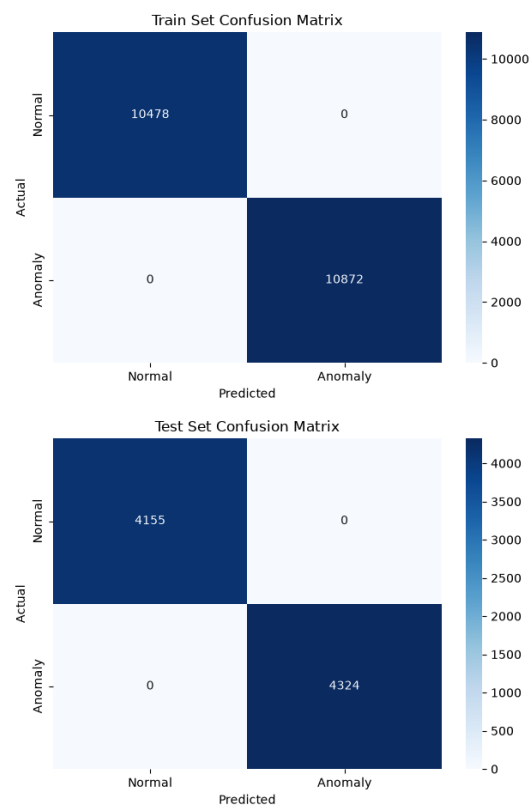


Figure 11: Confusion Matrix

Train set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	10478
Anomaly (1)	1.00	1.00	1.00	10872
accuracy			1.00	21350
macro avg	1.00	1.00	1.00	21350
weighted avg	1.00	1.00	1.00	21350

Test set evaluation: int8 model				
	precision	recall	f1-score	support
Normal (0)	1.00	1.00	1.00	4155
Anomaly (1)	1.00	1.00	1.00	4324
accuracy			1.00	8479
macro avg	1.00	1.00	1.00	8479
weighted avg	1.00	1.00	1.00	8479

Figure 12: Classification Report (int8 model)

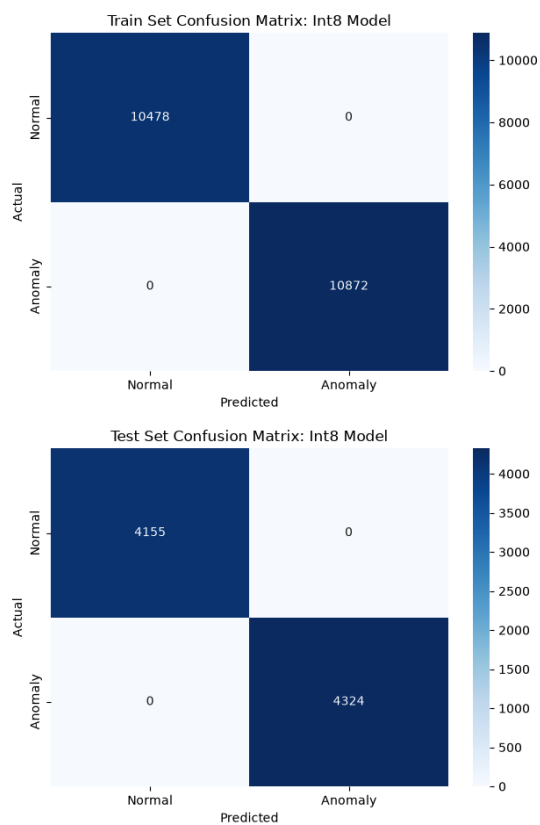


Figure 13: Confusion Matrix (int8 model)

Arduino Deployment

```
Reconstruction error: 0.148217  
Result: Normal activity  
Reconstruction error: 0.148216  
Result: Normal activity  
Reconstruction error: 0.148222  
Result: Normal activity  
Reconstruction error: 0.148223  
Result: Normal activity  
Reconstruction error: 0.148216  
Result: Normal activity  
Reconstruction error: 0.148217  
Result: Normal activity  
Reconstruction error: 0.148215  
Result: Normal activity  
Reconstruction error: 0.148215  
Result: Normal activity  
Reconstruction error: 0.148221  
Result: Normal activity  
Reconstruction error: 0.148229  
Result: Normal activity  
Reconstruction error: 0.148228  
Result: Normal activity  
Reconstruction error: 0.148231  
Result: Normal activity
```

Figure 14: Serial Output With Normal Activity (no motion)

```
Reconstruction error: 3.505157
Result: Anomaly detected
Reconstruction error: 3.500378
Result: Anomaly detected
Reconstruction error: 3.465742
Result: Anomaly detected
Reconstruction error: 3.427096
Result: Anomaly detected
Reconstruction error: 3.388696
Result: Anomaly detected
Reconstruction error: 3.406911
Result: Anomaly detected
Reconstruction error: 3.376086
Result: Anomaly detected
Reconstruction error: 3.313048
Result: Anomaly detected
Reconstruction error: 3.309556
Result: Anomaly detected
Reconstruction error: 3.274827
Result: Anomaly detected
Reconstruction error: 3.258539
Result: Anomaly detected
Reconstruction error: 3.149942
Result: Anomaly detected
Reconstruction error: 3.065159
Result: Anomaly detected
Reconstruction error: 3.021687
Result: Anomaly detected
Reconstruction error: 2.931002
Result: Anomaly detected
Reconstruction error: 2.918158
Result: Anomaly detected
Reconstruction error: 2.933121
Result: Anomaly detected
Reconstruction error: 2.858349
Result: Anomaly detected
```

Figure 15: Serial Output With Anomaly (motion)

Section 3: Discussion Questions

What threshold did you choose for anomaly detection, and why?

I chose 1.5 for my threshold. Referring to the error summary below, the maximum reconstruction error from the normal training and test datasets was about 1.37 with a maximum standard error of 0.003. The minimum reconstruction error from the anomalous train and test datasets was 2.5 with a maximum standard error of 0.62. 1.5 is at least one standard error above the maximum reconstruction error for normal data and at least one standard error below the minimum reconstruction error for the anomalous data.

Train normal	mean=0.098753, stderr=0.001669, min=0.008602, max=1.137708
Train anomaly	mean=42.216591, stderr=0.373219, min=2.495696, max=128.816559
Test normal	mean=0.136306, stderr=0.003891, min=0.009509, max=1.371906
Test anomaly	mean=42.531487, stderr=0.623820, min=2.839492, max=131.907028

Figure 16: Error Summary

How does reconstruction error help distinguish normal and anomalous activity?

The reconstruction error computes how much on average the autoencoder's output's components differ from the original input's components. If the autoencoder is trained on normal data, the reconstruction error should be relatively small. When an anomalous input is passed into the autoencoder, the reconstruction error would be relatively large compared to the typical errors with normal input data.

What information from the Python notebook must be preserved when deploying the model to Arduino?

We must ensure the reconstruction error threshold remains the same in both the notebook and the Arduino sketch.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The autoencoder model was trained assuming the inputs and outputs would be 300-dimensional vectors. Not only that, the training dataset was built by flattening 100 x 3 windows, where each column represents readings from one axis of the accelerometer, into 300-dimensional vectors. So the Arduino sketch must take care when marshaling in accelerometer readings from the window buffer into the input tensor of the autoencoder model. It also must take care to compare the correct values from the output with the original input when computing reconstruction errors.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The microcontroller is subject to the laws of motion and false anomalies can still be detected depending on where these measurements are taken. Consider the case where a person wearing the left ankle bracelet is sitting down while riding a car with seatbelts fastened. If the car brakes abruptly, the accelerometer will still detect anomalous activity even though the person wearing the bracelet may be sitting perfectly still. Another edge case is a person standing perfectly still while riding an elevator. The microcontroller will detect anomalous activity when the elevator starts and stops.